

 Stock & Foil

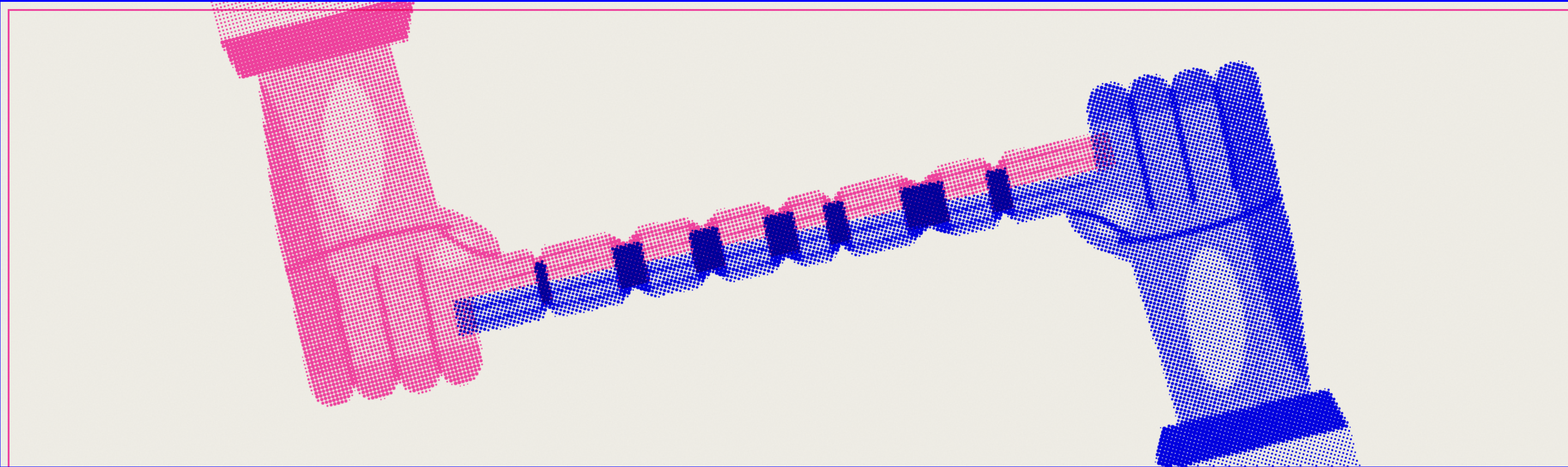
Private collateral registry for receivables finance · built on Midnight

Every invoice can be pledged once.

Prove it without revealing it.

A collateral registry that proves things it cannot read. Suppliers pledge invoices, financiers check them, and the register refuses a forged, an inflated or a second pledge of the same receivable.

The public ledger holds nothing but commitments, one-time markers and sealed records.



Stock, kept by the lender · Foil, kept by the buyer · notches cut from 0x31e5...46f7

Compact 0.31.1 · 12 circuits · 171 tests

Two lenders, one invoice, nobody able to check.

September 2025 · First Brands Group · Chapter 11
Source: Global Trade Review, on the receivables and inventory finance fraud alleged against the founder.

First Brands filed for Chapter 11 carrying roughly **\$2.3 billion** of receivables-finance liabilities. The invoices behind them are alleged to have been forged, inflated tenfold, and presented to more than one financier.

Three frauds, one shape: a receivable that is not what the lender was told it was, or is not the lender's to hold. On paper, each is indistinguishable from an honest one — and **none of the financiers could have caught it by comparing notes**, because there was no way to compare notes that cost less than the fraud.

\$2.3B

of receivables-finance liabilities at filing

10×

the multiple some invoice values were inflated by

>1

financiers shown the same invoice

Stock & Foil lets them check without showing each other anything.

Both fixes cost more than the fraud.

There are exactly two ways to check whether a receivable is already financed, and receivables finance has rejected both for the same reason.

Option one

Show the other lender your invoices

The diagram shows a flow from 'Lender A its book' to an 'invoice file' box, which then points to 'Lender B its book'. Below the 'invoice file' box, three red lines branch out, labeled 'client list', 'prices', and 'volumes', indicating the components of the shared data.

An invoice file is a client list and a price list. Handing it to a rival costs more than the fraud does, so every financier keeps its book to itself — and the same receivable gets funded twice.

Option two

Put every fingerprint in one registry

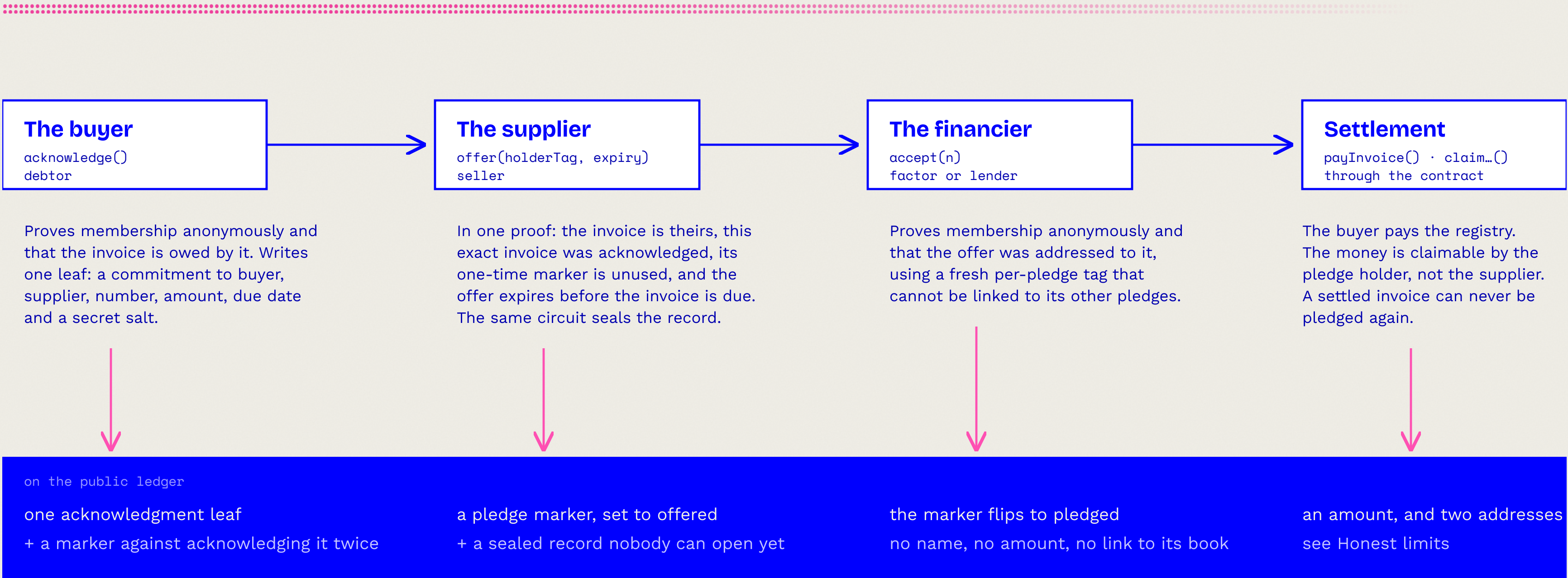
The diagram shows four boxes labeled 'financier' on the left, each with an arrow pointing to a single large pink box on the right labeled 'one operator holds every fingerprint'.

Central fingerprint registries do work, and they ask every financier to trust one operator with the shape of its whole book — who its clients are, how much it lends, and when.

Midnight's dual ledger removes the trade-off. The checks run in zero knowledge against private state, and the public ledger holds only what uniqueness needs: a commitment, a one-time marker, a sealed record.

Four parties, one record.

A receivable passes through four hands. Each hand proves something, and leaves a mark nobody else can read.



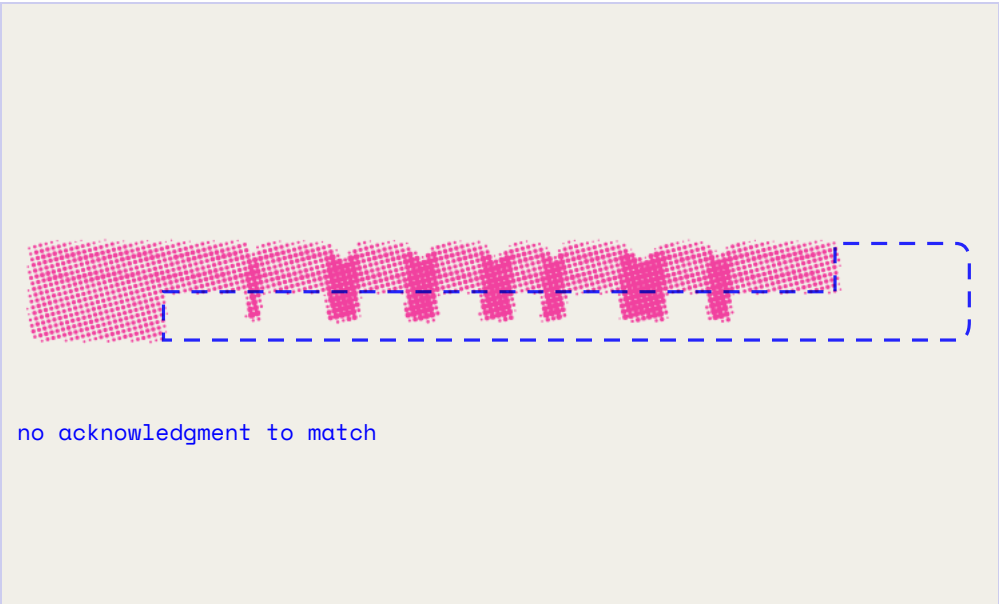
Every check runs while the transaction is being built. A fraudulent pledge cannot be proved, so it never reaches the chain at all — there is no bad state to roll back and no failed transaction to read.

Three refusals.

A tally stick was split so that one half could not be forged against the other. These are the same idea, proved rather than whittled. The assert message in the contract is the product's refusal code, and the SDK and the interface carry it through unchanged.

A forged invoice

Financing raised against an invoice the buyer never owed.



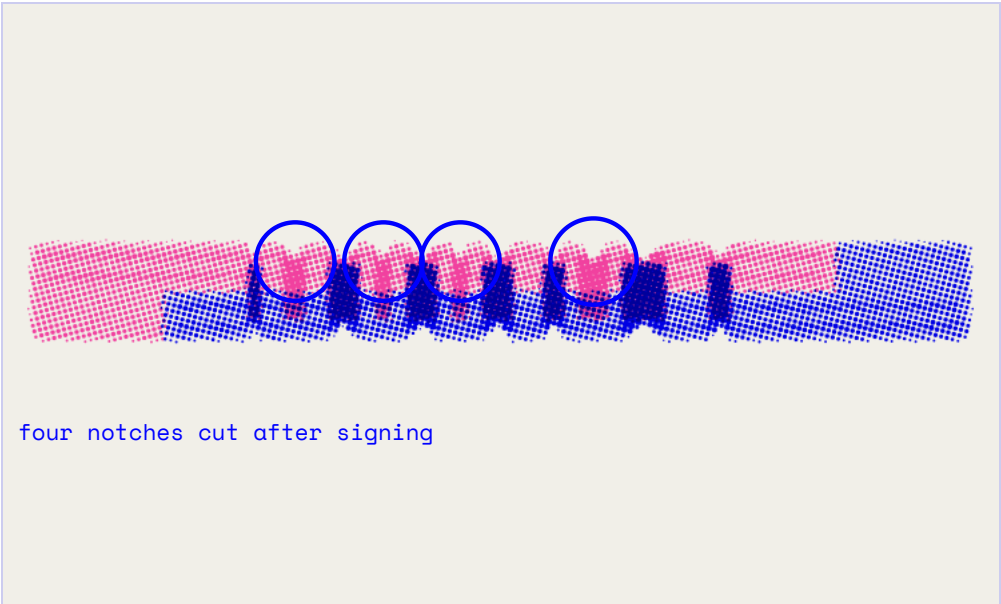
no acknowledgment to match

NOT_ACKNOWLEDGED

No acknowledgment leaf exists for this invoice, so the offer has no membership proof to make. There is no second half for it to match.

An inflated invoice

The buyer owes \$2,300. The financier is shown \$23,000.



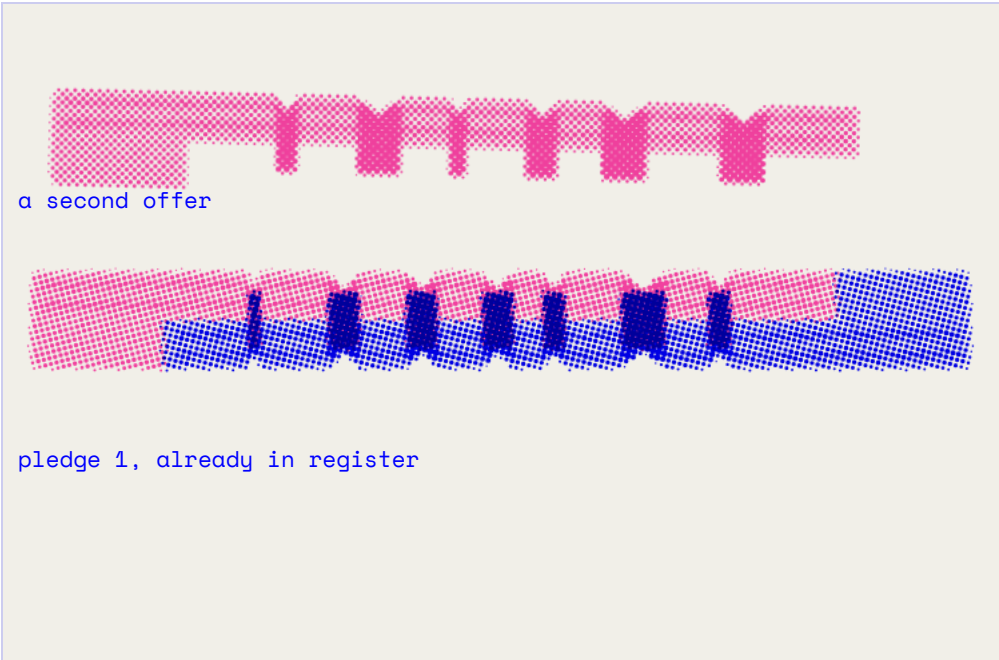
four notches cut after signing

NOT_ACKNOWLEDGED

The amount is inside what the buyer signed. A \$23,000 invoice is a different commitment from the \$2,300 on record, and nothing acknowledges it.

A double pledge

One receivable financed by several financiers at once.



a second offer

pledge 1, already in register

ALREADY_ENCUMBERED

The one-time marker for this receivable is already spent. A stick that has been split cannot be split again.

The refused lender learns exactly one bit: this receivable is already encumbered. Not by whom, not for how much, not since when — and the existing pledge is neither touched nor disclosed.

What the ledger shows, and what it never shows.

Printed on the ledger public

- **Acknowledgment leaves.** One per acknowledged invoice. The leaf does not say which.
- **Pledge markers and their status.** Offered, pledged, released or settled.
- **Per-pledge holder tags.** A fresh tag for every pledge.
- **Sealed records and their ephemeral keys.**
- **Certificate floors, pool size and validity.**
- **Disclosure requests and approvals.**
- **Settlement amounts and addresses,** unshielded.

Settlement is the one exception, and it is published under **Honest limits** rather than hidden.

Never printed stays with the parties

- **The invoice.** Buyer, supplier, number, amount, due date.
- **Which invoice a marker belongs to.**
- **Which financier holds which pledge,** or how big its book is.
- **The record contents,** without 2 of 3 keyholders.
- **The receivables inside a borrowing base.**
- **Anything the auditor learns,** to anyone else.
- **Who was refused,** and what they were trying to pledge.

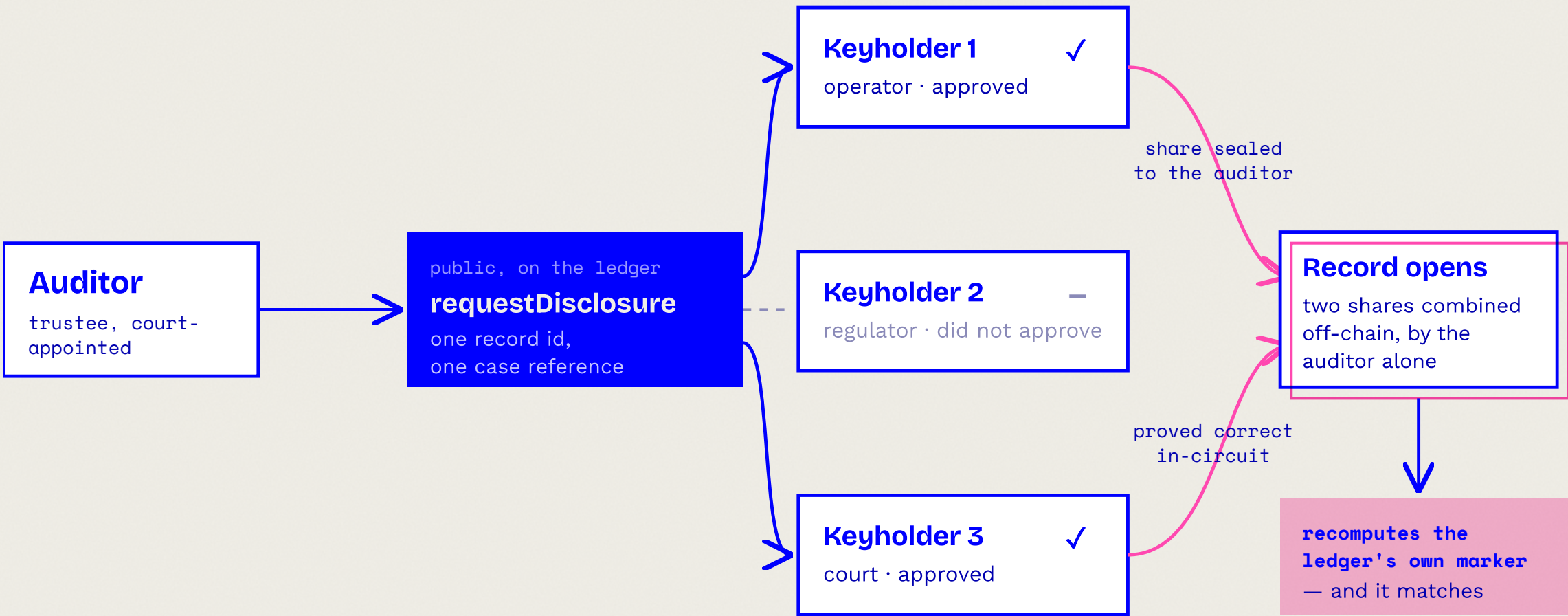
The supplier salts the invoice before the buyer acknowledges it, so a financier who guesses an invoice still cannot test whether it is encumbered.

The explorer in the app reads the same projection an indexer does — and this band is the whole of it. Counts, markers, statuses, ciphertexts. There is nothing underneath to be careless with.

1	2	0	0	0	0	0	0
Debtors	Financiers	Acknowledgments	Pledge markers	Sealed records	Certificates	Disclosure requests	Sealed shares

Disclosure nobody can do alone.

An insolvency practitioner has to be able to open a specific pledge. One party being able to do that would undo everything else.



the key

A 2-of-3 Shamir sharing of the disclosure key over the Jubjub scalar field. The constructor checks the sharing itself, in point form, and refuses a deployment whose ceremony would let one party read everything or no two parties read anything.

the share

A keyholder's share of the record key is computed inside the circuit and sealed to the auditor's key. No keyholder ever publishes anything another keyholder can use.

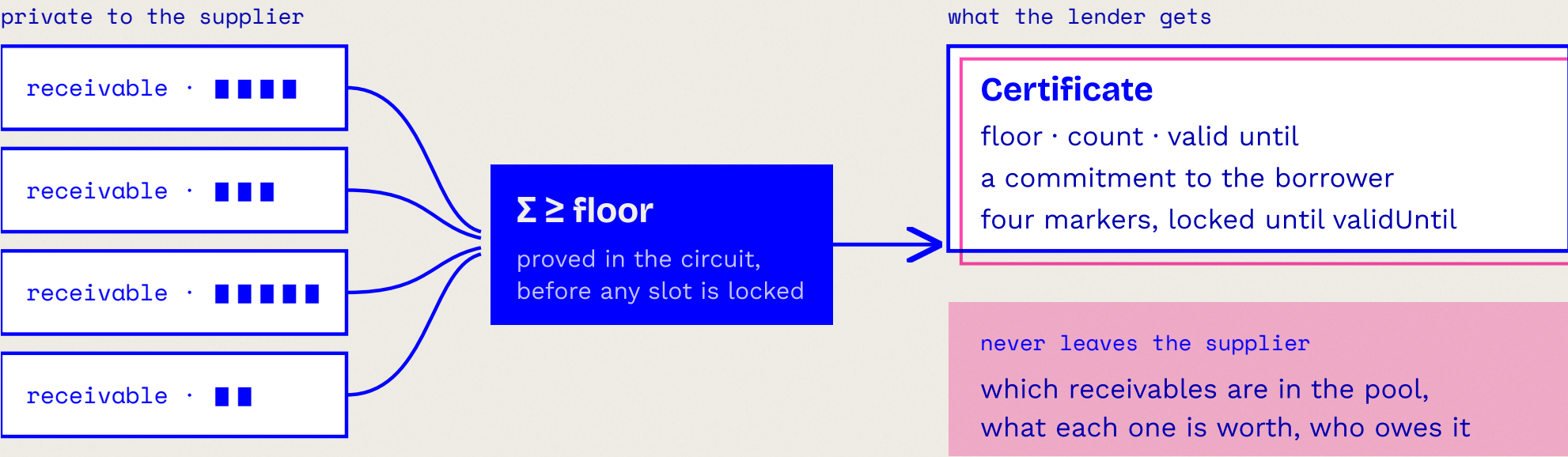
the check

The opened record recomputes the invoice fingerprint, the acknowledgment leaf and the pledge marker, so a doctored disclosure fails against the ledger's own value. The request and both approvals stay on the ledger: no record can be read without the reading being public.

Devnet run: approvals from keyholders 1 and 3 · the opened record recomputed marker 9450be64... and the ledger holds the same value.

Prove the pool clears the floor.

An asset-based lender advances against a borrowing base. Today it gets a spreadsheet of line items and takes the borrower's word for the rest.



Each locked slot becomes an ordinary offer expiring with the certificate, which the lender takes up with the same **accept** it uses for a single invoice. Distinctness and the floor are proved before any slot is locked, so a pool that lists one invoice twice is refused for the duplicate rather than for an encumbrance it caused itself.

the certificate's own refusals, asserted in this order

BAD_EXPIRY

DUPLICATE_CERTIFICATE

EMPTY_POOL

NOT_INVOICE_OWNER

NOT_ACKNOWLEDGED

INVOICE_OVERDUE

DUPLICATE_INVOICE

BELOW_FLOOR

ALREADY_SETTLED

ALREADY_ENCUMBERED

BAD_EPHEMERAL

Up to four receivables

Each proved acknowledged, unencumbered, not yet settled and not overdue — the same four checks a single pledge passes.

The lock, and what it does not prove

A certificate that only asserted a total would be a promise. This one writes four pledge markers in the same transaction. What no circuit can check is that those markers are addressed to the lender the certificate names: a holder tag is derived from the lender's own secret, so the contract has nothing to test the seller's claim against. The lender recomputes every tag from public state before money moves.

Kestrel Components.

Issues invoices with the salt that keeps them untestable, offers acknowledged receivables to one financier at a time, and certifies a borrowing base.

Your invoice book

Private to you. The registry never holds an invoice — only a hash of the whole document, which is why the salt below matters: without it, a financier who guesses an invoice still cannot test whether it is encumbered.

Invoice	Amount	Due	Acknowledged	Encumbrance	Pledge marker
SF-2026-1001 The receivable the whole replay turns on.	\$2,399.00	14 Dec 2026	no	free	—
SF-2026-1002 Held back for the borrowing base.	\$1,200.00	29 Nov 2026	no	free	—

What this party sees

■ Its own invoice book

■ Holder tags financiers gave it

■ Which of its receivables are encumbered

And what it does not

■ Another seller's book

■ Whether a refusal came from a

The supplier's own book, in the app. The registry never holds an invoice — only a hash of the whole document.

Twelve circuits, one contract.

Compact 0.31.1, language 0.23. Authorisation is always knowledge of a secret whose hash is on the ledger or inside committed data — never the caller's public key.



The dual ledger

public state, readable by anyone

- Two membership trees, depth 10
- The acknowledgment tree, depth 16, and its nullifier set
- Pledge markers, keyed by nullifier
- Sealed records, certificates, disclosure requests and shares
- The ceremony: disclosure key, three keyholder keys, auditor key

private state, never leaves the party

- The invoice and its salt
- Each party's secret, and the Merkle paths it proves with
- Holder tags, sealing scalars, keyholder shares

Exported circuits	12
Refusal codes, asserted in order	34
Domain separators, one per purpose	15
Compile with proving keys	1 m 54 s

Prover keys

SHA-256 dominates key size and proof time

certifyBorrowingBase	42.9 MB
offer	11.1 MB
acknowledge	10.0 MB
payInvoice	10.0 MB
accept	5.2 MB
claimAsSeller	5.2 MB
approveDisclosure	3.1 MB
requestDisclosure	3.1 MB
admitDebtor	2.8 MB
admitFinancier	2.8 MB
claimAsHolder	2.8 MB
release	2.8 MB

The seal

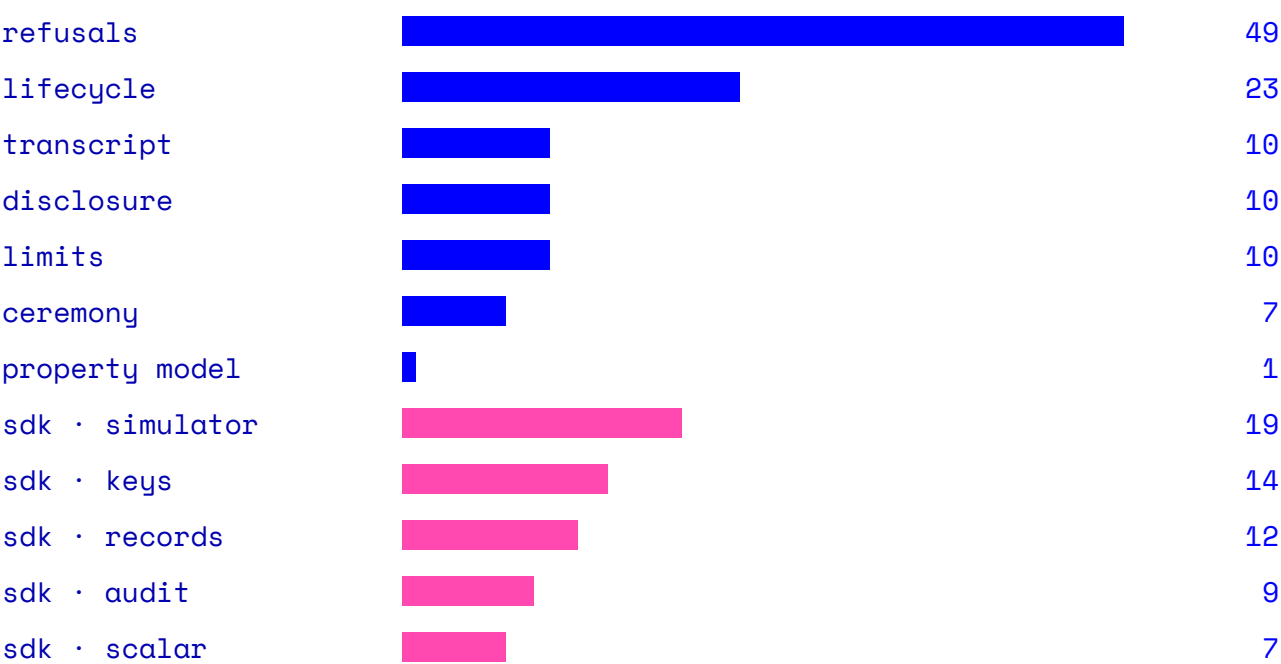
A fresh scalar e per record. $E = e \cdot G$ is published; the mask is $\text{Poseidon}(S.x, S.y, j)$ over $S = e \cdot \text{disclosurePk}$. Poseidon because a SHA-derived mask sits below 2^{248} and would leak the high bits of the 248-bit identities it pads. Records carry a version for that reason.

The deploy

Twelve verifier keys total about 26 kB, and one transaction's byte budget takes six. The deploy carries six, then six `VerifierKeyInsert` maintenance transactions carry the rest. Seven transactions, and no address is published until every circuit has a key.

171 tests, and the ones that are meant to fail.

On the simulator



and beyond it

Devnet integration, with real proofs	4
Playwright, the guided replay end to end	3

The simulator runs the compiled circuits without proving, so the whole suite finishes on a laptop with no Docker and no node. It is the same code path the browser sandbox uses, which is why the sandbox can refuse a fraud for real.

In the history, each behaviour is its own commit with the test that proves it — `test(contract)`: the public transcript, and the limits that are not enforced.

The adversarial suite

One scenario for every refusal code the contract can raise, plus the lying witness: when no real Merkle path exists the witness hands back a dummy one, so the refusal comes from the circuit's assert and not from a JavaScript exception. That distinction is the test.

The property model

fast-check drives random lifecycles against a reference model. The invariants: at most one active pledge per invoice, a settled invoice is never pledgeable again, and proceeds are paid exactly once to the correct payee.

Negative controls

Threshold decryption is tested with every pair of keyholders — and with one share alone, and with the wrong auditor key, both of which must fail. An opened record that does not recompute the ledger's marker is a failure, not a warning.

The documented leaks

A whole suite reproduces the leaks and grieving vectors the threat model admits, and another asserts value by value what each circuit publishes. "Not defended" is a measured statement rather than an assumption.

Show your working.

Nothing on this slide is typed by hand. It comes from the files the command-line tool wrote when it deployed the contract and ran the scenario against it.



Deployed

PREVIEW · PUBLIC TESTNET

Contract	9be70aa1...33e8f7
Deploy tx	d8732da6...d13f68
Block	879363
Verifier keys	+6 maintenance txs
Deployed	15 Sept 2026, 17:40 UTC
Explorer	preview.midnightexplorer.com

A public Midnight testnet anyone can query without asking us for anything. Seven transactions: a constructor carrying six verifier keys, then one maintenance transaction for each remaining circuit, because twelve keys do not fit in one.

And on a local devnet

the same 27 steps

A node, an indexer and a proof server in Docker on one laptop. It shows the code runs but proves nothing to a stranger, and **PROOF.md** labels it that way.

The run

PREVIEW

Steps	27
Transactions	20
Blocks	879534 → 879615
Refusals recorded	5
Disclosure	2 of 3, verified

Two **NOT_ACKNOWLEDGED**, then **ALREADY_ENCUMBERED**, **NOT_PAYEE** for a supplier reaching for pledged proceeds, and **BELOW_FLOOR** for an overstated pool. Each recorded as a refusal with no transaction to show for it, because there was none.

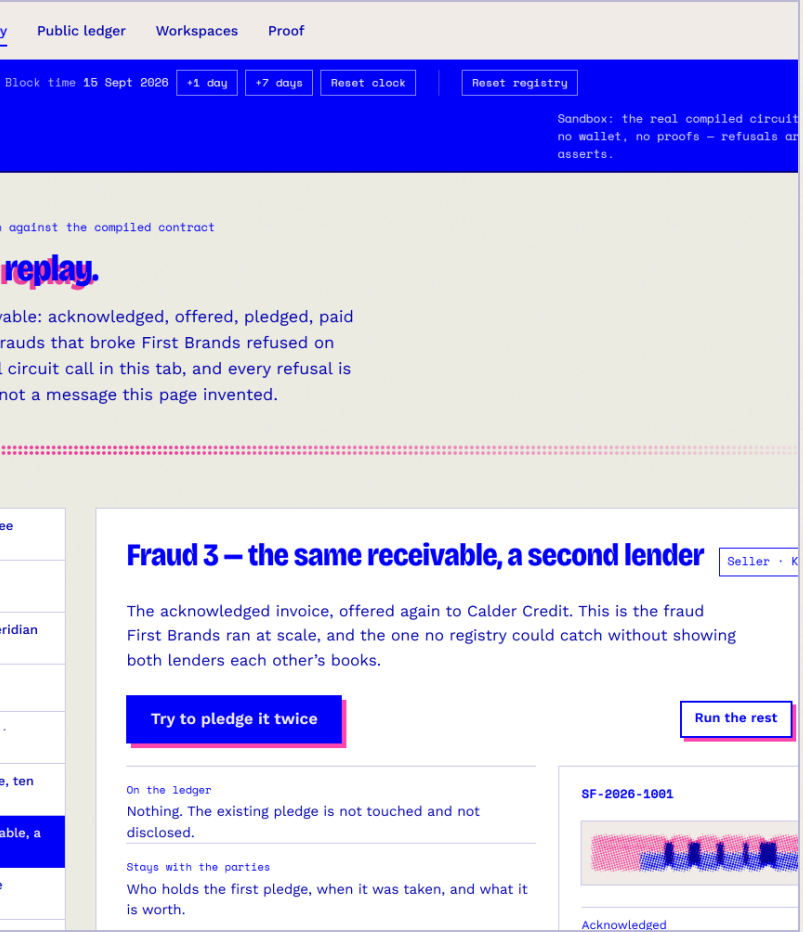
What a judge runs

```
npm test
171 tests, no Docker, no node

npm run verify:onchain
reads the deployed contract through
the public indexer and compares it
with the evidence, field by field.
No wallet, no proof server, and a
non-zero exit on any mismatch.
```

Zero-install sandbox

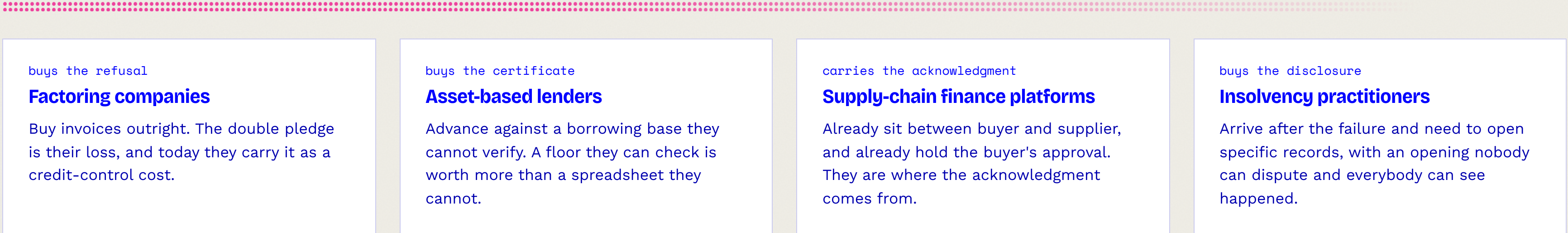
The web app runs the real compiled circuits in the browser tab. No wallet, no install, no chain. Every refusal on screen is the contract's own assert, not a message the page invented.



The double pledge, refused in the browser tab.

Who buys a refusal.

The product is not the ledger and not the proof. It is the moment a financier is told *no*, by a register that could not have told it anything else.



The adoption path runs through one platform, not fifty banks

01

A supply-chain finance platform integrates the SDK. Its own book becomes the first registry — useful on day one, with no counterparty needed to start, because it already double-finances itself by accident.

02

A second financier joins because the first one can now refuse what it cannot see. Neither has to show the other anything, which is the only reason the second one can join at all.

03

The consortium operator admits members after off-chain KYB. The contract gives that operator no power to read a record, take a pledge, settle one or open a disclosure.

Nothing in the design needs a second member on day one — which is the only way a registry like this ever gets a second member.

How it is paid for

- The consortium licenses the registry and runs the admission.
- The financier pays per receivable checked. The refusal is the unit.
- Disclosure is sold to the practitioners who need it, separately from the register.

Priced against the loss it prevents rather than the data it stores — because it stores none.

Honest limits.

Every item below has a test that reproduces it, so "not defended" is a measured statement rather than an assumption.

- **Collusion still works.** A buyer who colludes with a supplier can acknowledge an invoice that does not exist. The registry makes that acknowledgment non-repudiable; it does not prevent it.
 - **Settlement is public.** The settlement token is unshielded, so amounts and payer and payee addresses are visible at payment time. Contract custody of shielded coins is blocked upstream.
 - **The operator is trusted twice.** It decides who is admitted, and as the deployer it holds the contract's maintenance authority — a strictly larger power. Production should move that authority to a multi-party key, or burn it.
 - **The keys are dealt, not generated.** A ceremony splits the disclosure key rather than a distributed protocol, and two colluding keyholders reconstruct it off-chain with no public request at all.
 - **Approvals are per record, not per case.** A keyholder's share is bound to the record, so two requests naming one record with one approval each open it. The threshold holds; the per-request trail does not.
 - **A certificate names a lender it cannot bind.** The holder tags on its locked slots come from the seller, and no circuit can tie them to that lender. The lender recomputes every tag from public state before it advances.
 - **A certificate shows its markers.** A borrowing base publishes the markers of the invoices it locks. That is exactly what makes the lock verifiable.
- **A seller can leak its own records.** Reusing a sealing scalar makes two records comparable to anyone, and re-offering with the same scalar overwrites the earlier record. The SDK draws a fresh one every time and can spot a repeat in public state — the discipline is still the caller's.
 - **A settled, unfinanced book is linkable.** An opened record reveals the supplier's id, and the seller payee tag is derived from it, so every settled-but-unfinanced receivable of that supplier becomes recognisable without any further approval.
 - **Timing correlates.** An observer can link an acknowledgment to a later offer by when each happened.
 - **Sealing rests on Poseidon.** The platform does not promise the hash stays stable across upgrades, which is why every record carries a version.
 - **Test networks only.** Mainnet contract deployment on Midnight is gated by the Foundation's review process.
 - **Not audited.** There has been an internal security review, with findings accepted on the record. There has been no external audit.

The threat model names the attacker behind each of these, the property it breaks and the test that reproduces it. Both documents ship with the repository.

[docs/THREAT-MODEL.md](#) · [docs/PRIVACY-BOUNDARY.md](#)
[contract/test/limits.test.ts](#) · [contract/test/transcript.test.ts](#)

Three waves, then a consortium.

The contract is close to finished. What is not finished is the part that makes a registry worth joining: the other members.

Wave 1 · shipped

The register, and the three refusals

- Twelve circuits, one contract
- 171 tests, 49 of them adversarial
- 2-of-3 disclosure that self-verifies
- Borrowing base that locks what it certifies
- Settlement through the contract
- A sandbox that needs no wallet
- A deployment on a public testnet
- A full lifecycle run with real proofs

Everything a first member needs to get value out of the registry on its own.

Wave 2 · next

Nobody should need to hold a token to be checked

- Fee sponsorship, so participants need no DUST
- In-browser proving through 1AM
- Batch acceptance for a financier's book
- Distributed key generation for the disclosure key
- Bulk disclosure for an insolvency

The friction that stops a real financier from trying it: a token to hold, a prover to install, one pledge at a time.

Wave 3 · after

Where the invoices already are

- Ledger-9 migration for contract events
- ECDSA-signed e-invoices straight from ERP systems
- Shielded settlement, once contract custody lands
- A mainnet deployment request
- An adapter for existing factoring platforms

The integrations that make the registry the place a pledge gets recorded, rather than a second system to remember.

A register with one member is a private database. The work after Wave 3 is the consortium — the operator, the admission rules and the second financier — not the contract.

Every invoice can be pledged once.
Prove it without revealing it.